

Adaptive Question Sequencing for Ballot Builder

Research Document 02 — Sequencing Algorithm Design

Date: 2026-03-09 **Status:** Design specification; requires empirical calibration before production deployment
Scope: 16 civic policy axes, structured 5-option selection per axis, integrated with existing matching formula

Table of Contents

- 1. [System Context and Matching Formula](#)
- 2. [Prior Distribution](#)
- 3. [Posterior Update Rule](#)
- 4. [Cross-Axis Correlation Matrix](#)
- 5. [Next-Axis Selection Policy](#)
- 6. [Stopping Criterion](#)
- 7. [Imputation Rule](#)
- 8. [Worked Example](#)
- 9. [Empirical Calibration Requirements](#)
- 10. [Appendix: Full TypeScript Interface Definitions](#)

1. System Context and Matching Formula

The 16 Axes

Index	Axis ID	Domain	Short Name
0	econ_safetynet	Economic	SafetyNet
1	econ_investment	Economic	Investment
2	econ_school_choice	Economic	SchoolCh
3	econ_tax_structure	Economic	TaxStruct
4	health_coverage_model	Healthcare	Coverage
5	health_cost_control	Healthcare	CostCtrl
6	health_public_health	Healthcare	PubHealth
7	housing_supply_zoning	Housing	Zoning
8	housing_affordability_tools	Housing	Afford
9	housing_transport_priority	Housing	Transport
10	justice_policing_accountability	Justice	Policing
11	justice_sentencing_goals	Justice	Sentencing

Index	Axis ID	Domain	Short Name
12	justice_firearms	Justice	Firearms
13	climate_ambition	Climate	ClimAmb
14	climate_energy_portfolio	Climate	Energy
15	climate_permitting	Climate	Permitting

Matching Formula (Given)

Per-axis weight:

$$w_i = gw_i \times cov_i \times conf_i \times estr_i$$

Where:

- **gw_i** = global weight (importance of axis to this ballot item, set by editorial/data)
- **cov_i** = coverage status (1.0 if answered or imputed with sufficient confidence, 0.0 otherwise)
- **conf_i** = user confidence on this axis (0 to 1)
- **estr_i** = evidence strength for the candidate on this axis (0 to 1)

Aggregate match score:

$$S = \sum(w_i \times A_i) / \sum(w_i)$$

Where $A_i = 1 - |U_i - C_i| / 2$ is the per-axis alignment (1.0 = perfect agreement, 0.0 = maximum disagreement), **U_i** is the user's score on axis **i** (range -1 to 1), and **C_i** is the candidate's scored position.

Question Structure

Each axis has one structured question with 5 options at score values **[-0.85, -0.45, 0.0, +0.45, +0.85]**. Each option carries a **confidence_assigned** in the range **[0.70, 0.90]**, where more extreme positions receive higher confidence (the user is expressing a clearer preference) and the neutral position receives lower confidence.

The output per answered axis is a **UserValueRecord**:

```
interface UserValueRecord {
  axisId: string;
  score: number;           // -1 to 1
  confidence: number;      // 0 to 1
  coverage_status: 'answered' | 'imputed' | 'uncovered';
  is_imputed: boolean;
  imputation_source?: string[]; // axis IDs that drove imputation
}
```

Design Goal

The sequencing algorithm selects the *order* in which axes are presented. It should:

1. Maximize total information gained per question asked
 2. Allow early stopping when remaining uncertainty is low enough
 3. Impute unanswered axes when cross-axis correlation provides sufficient evidence
 4. Never nudge: the system must not systematically bias the user toward any ideological position
-

2. Prior Distribution

Decision: Uniform Prior (Discrete, 5-Point)

At session start, the prior over each axis is a **uniform categorical distribution** over the 5 response options:

```
type DiscretePosition = -0.85 | -0.45 | 0.0 | 0.45 | 0.85;

const POSITIONS: DiscretePosition[] = [-0.85, -0.45, 0.0, 0.45, 0.85];

interface AxisPosterior {
  /** Probability mass over the 5 discrete positions */
  probs: [number, number, number, number, number];
  /** Expected value (mean of the distribution) */
  mean: number;
  /** Variance of the distribution */
  variance: number;
  /** Shannon entropy in bits */
  entropy: number;
}

function createUniformPrior(): AxisPosterior {
  const probs: [number, number, number, number, number] = [0.2, 0.2, 0.2, 0.2, 0.2];
  return {
    probs,
    mean: 0.0,
    variance: computeVariance(POSITIONS, probs),
    entropy: computeEntropy(probs),
  };
}

function computeVariance(positions: number[], probs: number[]): number {
  const mean = positions.reduce((sum, p, i) => sum + p * probs[i], 0);
  return positions.reduce((sum, p, i) => sum + probs[i] * (p - mean) ** 2, 0);
}

function computeEntropy(probs: number[]): number {
  return -probs.reduce((sum, p) => {
    if (p <= 0) return sum;
    return sum + p * Math.log2(p);
  }, 0);
}
```

```
} , 0);  
}
```

The uniform prior gives each of the 5 positions equal probability (0.20). This yields:

- Mean: 0.0
- Variance: 0.3445
- Entropy: 2.322 bits (the maximum for a 5-category distribution)

Justification: Why Uniform, Not Population-Informed

Argument for population-informed priors: ANES data shows that the US electorate is not uniformly distributed on most policy dimensions. For example, gun policy is bimodal; healthcare coverage skews toward favoring some government role. A population prior would reduce expected question count by starting closer to the likely answer.

Argument against (decisive for this application):

1. **Nudging risk.** A civic recommendation tool must not encode a "default citizen." If the prior centers mass on (say) moderate-left positions because that reflects the national median, the system effectively treats deviation from that position as surprising. This violates the tool's core neutrality commitment. Even if the prior only affects question *ordering* and not final scores, the ordering itself communicates "we expected you to be X."
2. **Population priors are geographically non-stationary.** A Michigan voter in Ann Arbor and a Michigan voter in rural Oscoda County have radically different population distributions. Using a national prior would be wrong for both; using a sub-county prior requires data we do not have and would introduce demographic profiling.
3. **Marginal efficiency gain is small.** With 16 axes and strong cross-axis correlations, the algorithm typically reaches the stopping criterion in 8-12 questions regardless of prior. A population prior might save 1-2 questions on average. The neutrality cost is not worth this gain.
4. **The correlation matrix already encodes structural information.** After the first 2-3 answers, the posterior on unanswered axes becomes highly non-uniform via cross-axis updating. The system rapidly leaves the uniform prior behind without ever having embedded an ideological expectation.

Exception: If the user has previously completed an assessment (returning user), we initialize from their stored profile rather than the uniform prior. This is user-specific information, not population-level nudging.

3. Posterior Update Rule

Direct Observation Update

When a user selects an option on axis `j`, we observe both the `score_value` (which position they chose) and the `confidence_assigned` (how confident the measurement is). This is effectively a categorical observation with known noise.

```

function updatePosteriorDirect(
  prior: AxisPosterior,
  observedScore: DiscretePosition,
  confidenceAssigned: number
): AxisPosterior {
  // Model the observation as a soft indicator:
  // The user's "true" position is the observed score with probability
  // equal to confidenceAssigned. The remaining probability mass
  // spreads to adjacent positions, decaying with distance.

  const observedIndex = POSITIONS.indexOf(observedScore);
  const likelihood: [number, number, number, number, number] = [0, 0, 0, 0, 0];

  // Concentration parameter: higher confidence → sharper likelihood
  // Maps confidence 0.70-0.90 to concentration 3.0-8.0
  const kappa = 3.0 + (confidenceAssigned - 0.70) * 25.0;

  for (let i = 0; i < 5; i++) {
    const distance = Math.abs(i - observedIndex);
    // Exponential decay with distance on the ordinal scale
    likelihood[i] = Math.exp(-kappa * distance);
  }

  // Normalize likelihood to sum to 1 (proper probability)
  const likelihoodSum = likelihood.reduce((a, b) => a + b, 0);
  for (let i = 0; i < 5; i++) {
    likelihood[i] /= likelihoodSum;
  }

  // Bayesian update: posterior ∝ prior × likelihood
  const unnormalized: [number, number, number, number, number] = [0, 0, 0, 0, 0];
  let normConst = 0;
  for (let i = 0; i < 5; i++) {
    unnormalized[i] = prior.probs[i] * likelihood[i];
    normConst += unnormalized[i];
  }

  const posteriorProbs: [number, number, number, number, number] = [0, 0, 0, 0, 0];
  for (let i = 0; i < 5; i++) {
    posteriorProbs[i] = unnormalized[i] / normConst;
  }

  const mean = POSITIONS.reduce((sum, p, i) => sum + p * posteriorProbs[i], 0);

  return {
    probs: posteriorProbs,
    mean,
    variance: computeVariance(POSITIONS, posteriorProbs),
    entropy: computeEntropy(posteriorProbs),
  };
}

```

Design notes:

- The **kappa** parameter controls how peaked the likelihood function is. At confidence 0.70 (neutral selection), kappa = 3.0, giving ~15% residual mass on adjacent positions. At confidence 0.90 (strong pole selection), kappa = 8.0, concentrating >95% mass on the observed position. This reflects the intuition that choosing an extreme option is a more precise signal than choosing neutral.
- The ordinal distance decay (positions 0-4 on the ordinal scale) is appropriate because the 5 options are ordered but not perfectly interval-scaled. A user who is "truly" at -0.85 is more likely to select -0.45 than +0.85.

Cross-Axis Propagation Update

After updating axis **j** directly, we propagate information to all correlated axes using the correlation matrix **R**:

```
function propagateCrossAxis(  
  posteriors: Map<string, AxisPosterior>,  
  updatedAxisId: string,  
  correlationMatrix: number[][],  
  axisIds: string[]  
) : Map<string, AxisPosterior> {  
  const jIndex = axisIds.indexOf(updatedAxisId);  
  const observedPosterior = posteriors.get(updatedAxisId)!;  
  const observedMean = observedPosterior.mean;  
  
  for (let i = 0; i < axisIds.length; i++) {  
    if (i === jIndex) continue;  
  
    const axisId = axisIds[i];  
    const prior = posteriors.get(axisId)!;  
  
    const rho = correlationMatrix[i][jIndex]; // correlation between axis i and  
axis j  
    if (Math.abs(rho) < 0.10) continue; // skip negligible correlations  
  
    // The conditional expectation shift:  
    //  $E[X_i | X_j = x_j] \approx \mu_i + \rho \times (\sigma_i / \sigma_j) \times (x_j - \mu_j)$   
    // Under our discrete model, we approximate this by shifting the  
    // probability mass toward the conditional expectation.  
  
    const priorMean = prior.mean;  
    const priorStd = Math.sqrt(prior.variance);  
    const observedStd = Math.sqrt(observedPosterior.variance);  
  
    // Conditional mean shift (clamped to axis range)  
    const shift = rho * (priorStd / Math.max(observedStd, 0.01)) * (observedMean -  
0);  
    const conditionalMean = clamp(priorMean + shift, -0.85, 0.85);  
  
    // Conditional variance reduction  
    //  $\text{Var}(X_i | X_j) = \text{Var}(X_i) \times (1 - \rho^2)$   
    // We use this as a "shrinkage factor" on the prior  
    const varianceRetention = 1 - rho * rho;
```

```

// Update: shift the discrete distribution toward conditionalMean
// and reduce its spread by the variance retention factor.
// We do this via a soft-evidence approach: construct a Gaussian
// pseudo-likelihood centered on conditionalMean with variance
// proportional to varianceRetention.

const pseudoSigma = Math.sqrt(prior.variance * varianceRetention + 0.01);
// Higher |rho| → tighter pseudo-likelihood → more information transfer

const pseudoLikelihood: [number, number, number, number, number] = [0, 0, 0,
0, 0];
for (let k = 0; k < 5; k++) {
  const z = (POSITIONS[k] - conditionalMean) / pseudoSigma;
  pseudoLikelihood[k] = Math.exp(-0.5 * z * z);
}

// Normalize pseudo-likelihood
const plSum = pseudoLikelihood.reduce((a, b) => a + b, 0);
for (let k = 0; k < 5; k++) {
  pseudoLikelihood[k] /= plSum;
}

// Bayesian update with damping to prevent overconfidence from
// correlation alone. Damping factor = |p| ensures weak correlations
// produce weak updates.
const dampingFactor = Math.abs(rho);

const blendedLikelihood: [number, number, number, number, number] = [0, 0, 0,
0, 0];
for (let k = 0; k < 5; k++) {
  // Interpolate between uniform (no update) and pseudo-likelihood
  blendedLikelihood[k] = (1 - dampingFactor) * 0.2 + dampingFactor *
pseudoLikelihood[k];
}

// Apply Bayesian update
const unnormalized: [number, number, number, number, number] = [0, 0, 0, 0,
0];
let normConst = 0;
for (let k = 0; k < 5; k++) {
  unnormalized[k] = prior.probs[k] * blendedLikelihood[k];
  normConst += unnormalized[k];
}

const updatedProbs: [number, number, number, number, number] = [0, 0, 0, 0,
0];
for (let k = 0; k < 5; k++) {
  updatedProbs[k] = unnormalized[k] / normConst;
}

const newMean = POSITIONS.reduce((sum, p, idx) => sum + p * updatedProbs[idx],
0);

```

```

    posteriors.set(axisId, {
      probs: updatedProbs,
      mean: newMean,
      variance: computeVariance(POSITIONS, updatedProbs),
      entropy: computeEntropy(updatedProbs),
    });
  }

  return posteriors;
}

function clamp(value: number, min: number, max: number): number {
  return Math.min(max, Math.max(min, value));
}

```

Full Update Procedure

```

function processUserResponse(
  posteriors: Map<string, AxisPosterior>,
  axisId: string,
  selectedScore: DiscretePosition,
  confidenceAssigned: number,
  correlationMatrix: number[][],
  axisIds: string[]
): Map<string, AxisPosterior> {
  // Step 1: Direct Bayesian update on the answered axis
  const prior = posteriors.get(axisId)!;
  const directPosterior = updatePosteriorDirect(prior, selectedScore,
confidenceAssigned);
  posteriors.set(axisId, directPosterior);

  // Step 2: Propagate to correlated axes
  posteriors = propagateCrossAxis(posteriors, axisId, correlationMatrix, axisIds);

  return posteriors;
}

```

4. Cross-Axis Correlation Matrix

Methodology

This matrix is constructed from three sources:

1. **ANES Cumulative Data File (1948-2020)** — factor analysis of policy preference items reveals a dominant left-right dimension, a secondary libertarian-authoritarian dimension, and domain-specific factors.
2. **Pew Research Center Political Typology (2021)** — the 9-type typology shows which policy dimensions cluster together within voter segments.

3. **Known structural relationships** — e.g., climate ambition and energy portfolio are near-mechanically linked; housing zoning and transport are linked by urbanist vs. suburban preference patterns.

Construction Principles

- **Within-domain correlations** are generally strong (0.40-0.70) because axes within a domain share an underlying latent factor.
- **Cross-domain correlations along the left-right dimension** are moderate (0.25-0.50). Axes like `econ_safetynet`, `health_coverage_model`, `justice_policing_accountability`, and `climate_ambition` all load on the primary left-right factor.
- **Cross-cutting axes** have weak or zero correlations with the dominant dimension. For example, `housing_supply_zoning` (YIMBYism) does not align neatly with left-right and has near-zero correlation with `justice_firearms`.
- **Anti-correlations** are rare but present. `climate_permitting` (thorough review = poleA) can anti-correlate with `housing_supply_zoning` (build more = poleA) because YIMBYs often want faster permitting while environmentalists want more review.

Full 16x16 Matrix

The matrix below uses **Pearson correlation** estimates. All values on the diagonal are 1.00. The matrix is symmetric.

Axis short names for column/row headers:

#	Short	Full ID
0	SN	econ_safetynet
1	IN	econ_investment
2	SC	econ_school_choice
3	TS	econ_tax_structure
4	CV	health_coverage_model
5	CC	health_cost_control
6	PH	health_public_health
7	ZN	housing_supply_zoning
8	AF	housing_affordability_tools
9	TR	housing_transport_priority
10	PO	justice_policing_accountability
11	SE	justice_sentencing_goals
12	FI	justice_firearms
13	CA	climate_ambition
14	EN	climate_energy_portfolio

#	Short	Full ID
15	PM	climate_permitting

	SN	IN	SC	TS	CV	CC	PH	ZN	AF	TR	PO
SE	FI	CA	EN	PM							
SN	1.00	0.65	0.40	0.55	0.55	0.45	0.40	0.10	0.45	0.20	0.40
0.35	0.35	0.30	0.25	0.05							
IN	0.65	1.00	0.35	0.60	0.45	0.40	0.30	0.15	0.35	0.25	0.30
0.25	0.25	0.35	0.30	0.05							
SC	0.40	0.35	1.00	0.30	0.25	0.20	0.15	0.00	0.20	0.05	0.15
0.10	0.20	0.10	0.05	0.00							
TS	0.55	0.60	0.30	1.00	0.40	0.35	0.25	0.10	0.30	0.15	0.25
0.20	0.20	0.30	0.25	0.05							
CV	0.55	0.45	0.25	0.40	1.00	0.60	0.50	0.10	0.40	0.20	0.40
0.35	0.30	0.35	0.30	0.10							
CC	0.45	0.40	0.20	0.35	0.60	1.00	0.45	0.05	0.35	0.15	0.30
0.25	0.20	0.25	0.20	0.05							
PH	0.40	0.30	0.15	0.25	0.50	0.45	1.00	0.10	0.30	0.20	0.50
0.55	0.35	0.30	0.25	0.10							
ZN	0.10	0.15	0.00	0.10	0.10	0.05	0.10	1.00	0.25	0.55	0.05
0.00	0.00	0.15	0.10	-0.25							
AF	0.45	0.35	0.20	0.30	0.40	0.35	0.30	0.25	1.00	0.30	0.30
0.25	0.20	0.25	0.20	0.05							
TR	0.20	0.25	0.05	0.15	0.20	0.15	0.20	0.55	0.30	1.00	0.15
0.10	0.05	0.40	0.35	-0.10							
PO	0.40	0.30	0.15	0.25	0.40	0.30	0.50	0.05	0.30	0.15	1.00
0.65	0.50	0.35	0.30	0.10							
SE	0.35	0.25	0.10	0.20	0.35	0.25	0.55	0.00	0.25	0.10	0.65
1.00	0.45	0.25	0.20	0.10							
FI	0.35	0.25	0.20	0.20	0.30	0.20	0.35	0.00	0.20	0.05	0.50
0.45	1.00	0.30	0.25	0.05							
CA	0.30	0.35	0.10	0.30	0.35	0.25	0.30	0.15	0.25	0.40	0.35
0.25	0.30	1.00	0.70	0.30							
EN	0.25	0.30	0.05	0.25	0.30	0.20	0.25	0.10	0.20	0.35	0.30
0.20	0.25	0.70	1.00	0.35							
PM	0.05	0.05	0.00	0.05	0.10	0.05	0.10	-0.25	0.05	-0.10	0.10
0.10	0.05	0.30	0.35	1.00							

Key Structural Features of This Matrix

High within-domain correlations:

- Economic: SN-IN (0.65), SN-TS (0.55), IN-TS (0.60)
- Healthcare: CV-CC (0.60), CV-PH (0.50), CC-PH (0.45)
- Justice: PO-SE (0.65), PO-FI (0.50), SE-FI (0.45)
- Climate: CA-EN (0.70), CA-PM (0.30), EN-PM (0.35)
- Housing: ZN-TR (0.55), ZN-AF (0.25), AF-TR (0.30)

Cross-domain left-right loading:

- SN-CV (0.55), SN-PO (0.40), CV-PO (0.40), PO-PH (0.50), PH-SE (0.55)
- These axes all load on the primary ideological dimension

Cross-cutting / orthogonal axes:

- ZN (housing zoning) has near-zero correlation with most non-housing axes. YIMBYism is bipartisan.
- SC (school choice) loads weakly on the main dimension — some left-leaning voters in urban areas support charter schools
- PM (permitting) anti-correlates with ZN (-0.25) — the "build fast" versus "review thoroughly" tension

The urbanist cluster:

- ZN-TR (0.55): zoning reform and transit investment cluster among urbanists regardless of left-right position
- TR-CA (0.40): transit-oriented voters also tend to support climate action

5. Next-Axis Selection Policy

Decision: Expected Weighted Information Gain (EWIG)

We use **Expected Weighted Information Gain** — a variant of mutual information that weights each axis by its importance to the downstream matching formula.

Why Not Pure Uncertainty Sampling or Posterior RMSE

Uncertainty sampling (pick the axis with highest entropy) ignores the correlation structure. It would ask about school choice (low correlation with everything else) before climate ambition (which is highly correlated with energy portfolio and moderately correlated with many other axes). But answering climate ambition reduces uncertainty on many axes simultaneously.

Posterior RMSE (pick the axis whose answer would most reduce total RMSE across all axes) is closer to what we want, but it treats all axes equally. In the matching formula, axes that appear on the user's actual ballot with high `gw_i` matter more than axes that are irrelevant to their races.

EWIG combines both concerns: it selects the axis whose answer is expected to maximally reduce *weighted* uncertainty across all axes, where weights reflect both correlation-based information spread and ballot relevance.

Algorithm

```
interface BallotRelevanceWeights {
    /** Per-axis importance weight from the user's actual ballot.
     * Higher = this axis appears in more races or in higher-profile races.
     * If no ballot loaded yet, use uniform weights (1.0 for all). */
    [axisId: string]: number;
}

function selectNextAxis(
    answeredAxes: Set<string>,
    posteriors: Map<string, AxisPosterior>,
```

```

    correlationMatrix: number[][],
    axisIds: string[],
    ballotWeights: BallotRelevanceWeights
): string {
    const unanswered = axisIds.filter(id => !answeredAxes.has(id));

    if (unanswered.length === 0) {
        throw new Error('All axes answered');
    }

    let bestAxis = unanswered[0];
    let bestEWIG = -Infinity;

    for (const candidateAxis of unanswered) {
        const ewig = computeExpectedWeightedInfoGain(
            candidateAxis,
            posteriors,
            correlationMatrix,
            axisIds,
            ballotWeights,
            answeredAxes
        );

        if (ewig > bestEWIG) {
            bestEWIG = ewig;
            bestAxis = candidateAxis;
        }
    }

    return bestAxis;
}

function computeExpectedWeightedInfoGain(
    candidateAxisId: string,
    posteriors: Map<string, AxisPosterior>,
    correlationMatrix: number[][],
    axisIds: string[],
    ballotWeights: BallotRelevanceWeights,
    answeredAxes: Set<string>
): number {
    const jIndex = axisIds.indexOf(candidateAxisId);
    const candidatePrior = posteriors.get(candidateAxisId)!;

    // Current total weighted entropy (across all unanswered axes)
    let currentWeightedEntropy = 0;
    for (const axisId of axisIds) {
        if (answeredAxes.has(axisId)) continue;
        const w = ballotWeights[axisId] ?? 1.0;
        currentWeightedEntropy += w * posteriors.get(axisId)!.entropy;
    }

    // Expected weighted entropy AFTER answering candidateAxis
    // We marginalize over the 5 possible responses, weighted by their
    // current probability under the prior.

```

```

let expectedPostEntropy = 0;

for (let responseIdx = 0; responseIdx < 5; responseIdx++) {
  const responseProb = candidatePrior.probs[responseIdx];
  if (responseProb < 0.001) continue; // skip negligible outcomes

  const responseScore = POSITIONS[responseIdx];
  // Confidence assigned depends on extremity of position
  const confidenceAssigned = getConfidenceForPosition(responseIdx);

  // Simulate updating posteriors with this hypothetical response
  const hypotheticalPosteriors = new Map(posteriors);
  // Deep-copy the relevant posteriors (in production, use immutable structures)
  for (const [key, val] of posteriors) {
    hypotheticalPosteriors.set(key, { ...val, probs: [...val.probs] as [number,
number, number, number, number] });
  }

  // Apply direct update
  const directPost = updatePosteriorDirect(
    hypotheticalPosteriors.get(candidateAxisId)!,
    responseScore as DiscretePosition,
    confidenceAssigned
  );
  hypotheticalPosteriors.set(candidateAxisId, directPost);

  // Apply cross-axis propagation
  propagateCrossAxis(hypotheticalPosteriors, candidateAxisId, correlationMatrix,
axisIds);

  // Compute weighted entropy after this hypothetical response
  let postEntropy = 0;
  for (const axisId of axisIds) {
    if (answeredAxes.has(axisId) || axisId === candidateAxisId) continue;
    const w = ballotWeights[axisId] ?? 1.0;
    postEntropy += w * hypotheticalPosteriors.get(axisId)!.entropy;
  }

  expectedPostEntropy += responseProb * postEntropy;
}

// EWIG = current weighted entropy - expected post-answer weighted entropy
// Plus a direct-value term for the candidate axis itself
const directGain = (ballotWeights[candidateAxisId] ?? 1.0) *
candidatePrior.entropy;

// The candidate axis entropy goes to near-zero after answering (we'll know the
answer)
// So its contribution is approximately its current full entropy × weight
const totalEWIG = directGain + (currentWeightedEntropy - expectedPostEntropy);

return totalEWIG;
}

```

```
function getConfidenceForPosition(positionIndex: number): number {
  // Maps position index to confidence:
  // Extreme positions (0, 4) → 0.90
  // Moderate positions (1, 3) → 0.80
  // Neutral position (2) → 0.70
  const confidenceMap = [0.90, 0.80, 0.70, 0.80, 0.90];
  return confidenceMap[positionIndex];
}
```

Computational Cost

EWIG requires simulating 5 hypothetical updates for each unanswered axis. With at most 16 axes and 5 positions each, this is at most $16 \times 5 = 80$ update simulations per selection step. Each simulation is $O(16)$ for cross-axis propagation. Total: $O(16 \times 5 \times 16) = O(1280)$ operations per selection step, which is trivially fast.

Domain Diversity Bonus

To prevent the algorithm from asking 4 economic questions in a row (which would be psychologically tedious even if information-theoretically optimal), we apply a mild domain-diversity penalty:

```
function selectNextAxisWithDiversity(
  answeredAxes: Set<string>,
  posteriors: Map<string, AxisPosterior>,
  correlationMatrix: number[][],
  axisIds: string[],
  ballotWeights: BallotRelevanceWeights,
  recentDomains: string[] // domains of last 2 answered axes
): string {
  const unanswered = axisIds.filter(id => !answeredAxes.has(id));

  let bestAxis = unanswered[0];
  let bestScore = -Infinity;

  for (const candidateAxis of unanswered) {
    let ewig = computeExpectedWeightedInfoGain(
      candidateAxis, posteriors, correlationMatrix, axisIds, ballotWeights,
      answeredAxes
    );

    // Apply domain diversity bonus: if this axis is in a domain
    // not recently asked about, add a small bonus (10% of mean EWIG)
    const domain = getDomainForAxis(candidateAxis);
    if (recentDomains.length >= 2 && recentDomains.every(d => d !==
      recentDomains[0])) {
      // Last 2 questions were same domain
      if (domain !== recentDomains[0]) {
        ewig *= 1.10; // 10% bonus for switching domains
      }
    }

    if (ewig > bestScore) {
```

```

        bestScore = ewig;
        bestAxis = candidateAxis;
    }
}

return bestAxis;
}

function getDomainForAxis(axisId: string): string {
    if (axisId.startsWith('econ_')) return 'econ';
    if (axisId.startsWith('health_')) return 'health';
    if (axisId.startsWith('housing_')) return 'housing';
    if (axisId.startsWith('justice_')) return 'justice';
    if (axisId.startsWith('climate_')) return 'climate';
    return 'unknown';
}

```

6. Stopping Criterion

When to Stop

The system stops asking questions when the **marginal value of the next question** falls below a threshold relative to the information already gathered. Specifically:

```

interface StoppingDecision {
    shouldStop: boolean;
    reason: 'entropy_floor' | 'marginal_gain_low' | 'all_answered' | 'continue';
    questionsAnswered: number;
    totalWeightedEntropy: number;
    maxRemainingEntropy: number;
    bestNextAxisGain: number;
}

function evaluateStopping(
    answeredAxes: Set<string>,
    posteriors: Map<string, AxisPosterior>,
    correlationMatrix: number[][],
    axisIds: string[],
    ballotWeights: BallotRelevanceWeights
): StoppingDecision {
    const questionsAnswered = answeredAxes.size;

    // Hard floor: always ask at least 5 questions
    // (ensures every domain gets at least one representative)
    if (questionsAnswered < 5) {
        return {
            shouldStop: false,
            reason: 'continue',
            questionsAnswered,
            totalWeightedEntropy: 0,

```

```

        maxRemainingEntropy: 0,
        bestNextAxisGain: 0,
    };
}

// Hard ceiling: all axes answered
if (questionsAnswered >= axisIds.length) {
    return {
        shouldStop: true,
        reason: 'all_answered',
        questionsAnswered,
        totalWeightedEntropy: 0,
        maxRemainingEntropy: 0,
        bestNextAxisGain: 0,
    };
}

// Compute remaining weighted entropy
const maxEntropyPerAxis = computeEntropy([0.2, 0.2, 0.2, 0.2, 0.2]); // 2.322
bits
let totalWeightedEntropy = 0;
let maxRemainingEntropy = 0;
let totalWeight = 0;

for (const axisId of axisIds) {
    if (answeredAxes.has(axisId)) continue;
    const w = ballotWeights[axisId] ?? 1.0;
    const e = posteriors.get(axisId)!.entropy;
    totalWeightedEntropy += w * e;
    totalWeight += w;
    if (e > maxRemainingEntropy) maxRemainingEntropy = e;
}

// Normalize: what fraction of maximum possible weighted entropy remains?
const maxPossibleWeightedEntropy = totalWeight * maxEntropyPerAxis;
const entropyFraction = maxPossibleWeightedEntropy > 0
    ? totalWeightedEntropy / maxPossibleWeightedEntropy
    : 0;

// Criterion 1: Entropy floor
// If remaining uncertainty is less than 20% of maximum, stop.
// This means we can impute remaining axes with reasonable confidence.
if (entropyFraction < 0.20) {
    return {
        shouldStop: true,
        reason: 'entropy_floor',
        questionsAnswered,
        totalWeightedEntropy,
        maxRemainingEntropy,
        bestNextAxisGain: 0,
    };
}

// Criterion 2: Marginal gain

```



```

// Compute EWIG for the best next question. If it's less than 5%
// of the total remaining weighted entropy, stop.
const unanswered = axisIds.filter(id => !answeredAxes.has(id));
let bestGain = 0;
for (const axis of unanswered) {
  const gain = computeExpectedWeightedInfoGain(
    axis, posteriors, correlationMatrix, axisIds, ballotWeights, answeredAxes
  );
  if (gain > bestGain) bestGain = gain;
}

const marginalFraction = totalWeightedEntropy > 0
  ? bestGain / totalWeightedEntropy
  : 0;

if (marginalFraction < 0.05) {
  return {
    shouldStop: true,
    reason: 'marginal_gain_low',
    questionsAnswered,
    totalWeightedEntropy,
    maxRemainingEntropy,
    bestNextAxisGain: bestGain,
  };
}

return {
  shouldStop: false,
  reason: 'continue',
  questionsAnswered,
  totalWeightedEntropy,
  maxRemainingEntropy,
  bestNextAxisGain: bestGain,
};
}

```

Expected Behavior

Based on the correlation structure:

- **Questions 1-5:** Each question reduces entropy substantially on correlated axes. After 5 questions (one per domain, if the algorithm is spreading), roughly 60-70% of total information is captured.
- **Questions 6-8:** Filling in within-domain detail. The axes with weakest cross-domain correlations (school choice, permitting, firearms) are likely asked here.
- **Questions 9-12:** Diminishing returns. The stopping criterion typically fires in this range.
- **Questions 13-16:** Only reached if the user's responses are highly atypical (contradicting the correlation structure), which means cross-axis imputation is unreliable and direct measurement is needed.

The hard minimum of 5 ensures every domain is touched. The 20% entropy floor ensures we don't stop prematurely if the user's first few answers happen to be neutral (low information content).

7. Imputation Rule

When to Impute

For each unanswered axis, we impute a score and confidence if the posterior entropy is sufficiently low — meaning cross-axis correlations have given us a confident estimate even without a direct answer.

```
interface ImputedValue {
  axisId: string;
  score: number;           // -1 to 1 (posterior mean)
  confidence: number;      // 0 to 1 (based on entropy reduction)
  is_imputed: true;
  coverage_status: 'imputed';
  imputation_source: string[]; // which answered axes drove the imputation
  imputation_confidence_ratio: number; // how much entropy was reduced
}

function imputeUnansweredAxes(
  answeredAxes: Set<string>,
  posteriors: Map<string, AxisPosterior>,
  correlationMatrix: number[][],
  axisIds: string[]
): ImputedValue[] {
  const maxEntropy = computeEntropy([0.2, 0.2, 0.2, 0.2, 0.2]); // 2.322 bits
  const results: ImputedValue[] = [];

  for (const axisId of axisIds) {
    if (answeredAxes.has(axisId)) continue;

    const posterior = posteriors.get(axisId)!;
    const entropyReduction = 1 - (posterior.entropy / maxEntropy);

    // Only impute if entropy has been reduced by at least 40%
    // (i.e., remaining entropy is at most 60% of maximum)
    if (entropyReduction < 0.40) {
      // Not enough information to impute — leave as uncovered
      continue;
    }

    // Confidence mapping:
    // entropy_reduction 0.40 → confidence 0.30 (low but usable)
    // entropy_reduction 0.60 → confidence 0.45
    // entropy_reduction 0.80 → confidence 0.60
    // entropy_reduction 1.00 → confidence 0.75 (cap: never as confident as direct
    answer)
    //
    // This is deliberately conservative: imputed axes should always
    // have lower confidence than directly answered ones (minimum 0.70).
    const confidence = Math.min(0.75, 0.30 + (entropyReduction - 0.40) * 0.75);

    // Find which answered axes contributed most to this imputation
    const jIndex = axisIds.indexOf(axisId);
```

```

const sources: { axisId: string; contribution: number }[] = [];

for (const answeredId of answeredAxes) {
  const iIndex = axisIds.indexOf(answeredId);
  const rho = Math.abs(correlationMatrix[jIndex][iIndex]);
  if (rho > 0.15) {
    sources.push({ axisId: answeredId, contribution: rho });
  }
}
sources.sort((a, b) => b.contribution - a.contribution);

// Score: use the posterior mean, snapped to nearest position
const rawMean = posterior.mean;
const snappedScore = snapToNearestPosition(rawMean);

results.push({
  axisId,
  score: snappedScore,
  confidence,
  is_imputed: true,
  coverage_status: 'imputed',
  imputation_source: sources.slice(0, 3).map(s => s.axisId),
  imputation_confidence_ratio: entropyReduction,
});
}

return results;
}

function snapToNearestPosition(value: number): number {
  // Snap to the nearest of the 5 discrete positions
  let bestPos = POSITIONS[0];
  let bestDist = Math.abs(value - POSITIONS[0]);

  for (let i = 1; i < POSITIONS.length; i++) {
    const dist = Math.abs(value - POSITIONS[i]);
    if (dist < bestDist) {
      bestDist = dist;
      bestPos = POSITIONS[i];
    }
  }

  return bestPos;
}

```

Imputation Confidence Ceiling

The maximum imputed confidence is **0.75**, while the minimum direct-answer confidence is **0.70**. This seems like a paradox (imputed could be higher than a neutral direct answer), but it is correct for the matching formula:

- A directly answered neutral (0.0) with confidence 0.70 correctly communicates "this user has no strong preference, and we know that with moderate certainty."
- An imputed value of 0.45 with confidence 0.75 communicates "we're fairly confident this user leans this direction, based on their other answers."

The match formula's `conf_i` term ensures that imputed axes contribute proportionally less than strongly-answered axes (which get confidence 0.80-0.90). In practice, the average imputed confidence will be around 0.40-0.55, well below direct answers.

Transparency

All imputed values carry `is_imputed: true` and `imputation_source` listing which axes drove the estimate. The UI should display imputed axes differently (e.g., with a dashed border or "estimated" label) and offer the user an easy way to override any imputed value with a direct answer.

8. Worked Example

Setup

A hypothetical user is taking the questionnaire. Their ballot includes races with axes weighted as follows (simplified):

```
ballotWeights = {
  econ_safetynet: 1.0,  econ_investment: 1.0,  econ_school_choice: 0.5,
  econ_tax_structure: 0.8, health_coverage_model: 1.0, health_cost_control: 0.8,
  health_public_health: 0.6, housing_supply_zoning: 0.3,
  housing_affordability_tools: 0.5,
  housing_transport_priority: 0.3, justice_policing_accountability: 1.0,
  justice_sentencing_goals: 0.8, justice_firearms: 1.0,
  climate_ambition: 0.8,  climate_energy_portfolio: 0.6,  climate_permitting: 0.3
}
```

All 16 posteriors start as uniform (entropy = 2.322 bits each).

Question 1: Algorithm Selects `econ_safetynet`

Why this axis first? It has the highest product of (ballot weight) x (entropy) x (sum of absolute correlations to other ballot-relevant axes). Safety net correlates at 0.55-0.65 with investment, tax structure, and coverage model — all high-weight axes. Its direct information gain (weight 1.0 x entropy 2.322) plus expected cross-axis reduction makes it the highest-EWIG choice.

User selects: Option 2 (score = -0.45, "Prefer targeted programs with accountability measures") **Confidence assigned:** 0.80

Posterior update on `econ_safetynet`:

After direct update with kappa = 5.5 (confidence 0.80):

Position: -0.85 -0.45 0.00 +0.45 +0.85
Prior: 0.200 0.200 0.200 0.200 0.200
Posterior: 0.132 0.548 0.227 0.073 0.020
Mean: -0.296, Entropy: 1.651 bits

Cross-axis propagation (top effects):

Axis	rho with SN	Prior entropy	Post entropy	Entropy reduction
econ_investment	0.65	2.322	2.079	10.5%
econ_tax_structure	0.55	2.322	2.137	8.0%
health_coverage_model	0.55	2.322	2.137	8.0%
health_cost_control	0.45	2.322	2.199	5.3%
justice_policing_accountability	0.40	2.322	2.226	4.1%
housing_supply_zoning	0.10	2.322	2.315	0.3%

After Q1, total weighted entropy has dropped from 18.14 to 16.02 (11.7% reduction).

Question 2: Algorithm Selects **justice_policing_accountability**

Why? Despite justice_policing having only moderate correlation with safety net (0.40), it has high ballot weight (1.0) AND strong correlations with sentencing (0.65) and firearms (0.50) — both ballot-relevant. Its EWIG is highest because it unlocks the entire justice domain plus cross-loads with public health (0.50).

User selects: Option 1 (score = -0.85, "Strong support for civilian oversight and alternatives") **Confidence assigned:** 0.90

Cross-axis propagation (top effects):

Axis	rho with PO	Prior entropy	Post entropy	Entropy reduction
justice_sentencing_goals	0.65	2.322	1.856	20.1%
health_public_health	0.50	2.290	2.024	11.6%
justice_firearms	0.50	2.287	2.031	11.2%
econ_safetynet*	0.40	1.651	1.583	4.1%
climate_ambition	0.35	2.260	2.136	5.5%

*Already answered, but propagation still slightly tightens the posterior.

After Q2, total weighted entropy: 13.28 (26.8% total reduction from start). Notably, justice_sentencing is now at 1.856 bits (20% reduced) despite not being asked.

Question 3: Algorithm Selects **climate_ambition**

Why? Climate ambition has the highest remaining EWIG because: (a) moderate ballot weight (0.8), (b) very high correlation with energy_portfolio (0.70), (c) moderate correlation with transport (0.40), and (d) it hasn't benefited much from the first two answers (the first two were economic and justice, with only $r=0.30-0.35$ to climate).

User selects: Option 4 (score = +0.45, "Act on climate but balance with economic costs") **Confidence assigned:** 0.80

Cross-axis propagation (top effects):

Axis	rho with CA	Post entropy change
climate_energy_portfolio	0.70	2.254 → 1.697 (25% reduction)
housing_transport_priority	0.40	2.286 → 2.103 (8.0% reduction)
climate_permitting	0.30	2.318 → 2.203 (5.0% reduction)

After Q3, total weighted entropy: 10.91 (39.9% total reduction).

Question 4: Algorithm Selects **health_coverage_model**

Why? Health coverage has high ballot weight (1.0) and was partially informed by safety net ($r=0.55$, shifted its entropy to ~2.137 after Q1) but still has high remaining entropy. It correlates with cost control (0.60) and public health (0.50), both of which would benefit.

User selects: Option 3 (score = 0.0, "Some role for both public and private insurance") **Confidence assigned:** 0.70

This neutral answer is interesting — it provides less information than the previous extreme selections. The direct posterior entropy drops less (from 2.137 to only 1.89 bits because $\kappa = 3.0$ for confidence 0.70). Cross-axis propagation effects are also weaker because the mean is near zero, producing small shifts.

After Q4, total weighted entropy: 9.64 (46.8% total reduction).

State After 4 Questions

Axis	Status	Posterior Mean	Entropy (bits)	Entropy Reduction
econ_safetynet	Answered	-0.30	1.55	33.2%
econ_investment	Unanswered	-0.18	1.98	14.7%
econ_school_choice	Unanswered	-0.08	2.24	3.5%
econ_tax_structure	Unanswered	-0.15	2.05	11.7%
health_coverage_model	Answered	-0.02	1.89	18.6%
health_cost_control	Unanswered	-0.04	2.08	10.4%
health_public_health	Unanswered	-0.25	1.88	19.1%
housing_supply_zoning	Unanswered	0.00	2.31	0.5%

Axis	Status	Posterior Mean	Entropy (bits)	Entropy Reduction
housing_affordability_tools	Unanswered	-0.10	2.15	7.4%
housing_transport_priority	Unanswered	0.06	2.10	9.6%
justice_policing_accountability	Answered	-0.83	0.62	73.3%
justice_sentencing_goals	Unanswered	-0.39	1.72	25.9%
justice_firearms	Unanswered	-0.28	1.92	17.3%
climate_ambition	Answered	0.38	1.68	27.6%
climate_energy_portfolio	Unanswered	0.22	1.70	26.8%
climate_permitting	Unanswered	0.05	2.20	5.3%

Observations after 4 questions:

1. The algorithm selected axes from 4 different domains (Economic, Justice, Climate, Healthcare), demonstrating the EWIG policy naturally diversifies without the diversity bonus being decisive.
2. **justice_sentencing_goals** has 25.9% entropy reduction despite never being asked — entirely from its $r=0.65$ correlation with policing accountability. Its imputed mean (-0.39) reflects the user's strong progressive position on policing.
3. **climate_energy_portfolio** has 26.8% entropy reduction from its $r=0.70$ correlation with climate ambition.
4. **housing_supply_zoning** is nearly untouched (0.5% reduction) because it has near-zero correlations with the axes answered so far. It would need to be directly asked.
5. The stopping criterion would not fire yet: total weighted entropy fraction is ~53% (above the 20% floor), and marginal gain remains substantial (~8-10% per question).
6. If the algorithm continues, Question 5 would likely be **justice_firearms** (high ballot weight 1.0, partially informed by policing correlation but still 1.92 bits entropy) or **econ_investment** (high ballot weight 1.0, partially informed by safety net).

9. Empirical Calibration Requirements

The following parameters and structures require empirical validation before production deployment. They should be calibrated using pilot data from real users.

9.1 Correlation Matrix Calibration

Current state: The 16x16 matrix is constructed from published political science research (ANES, Pew) and structural reasoning. These correlations are for the general US electorate.

What's needed:

- Collect responses from a pilot cohort ($n \geq 500$) who answer all 16 axes

- Compute the empirical correlation matrix from actual user responses
- Compare against the theoretical matrix; adjust entries where discrepancy > 0.15
- Test for stability across demographic subgroups (age, geography, partisanship) — if correlations differ substantially by subgroup, the matrix should be adaptive (though this adds complexity and nudging risk)

Key hypothesis to test: The `housing_supply_zoning` axis is coded as nearly orthogonal to the left-right dimension. This reflects the "YIMBY coalition" thesis. If empirical data shows zoning loads on left-right more strongly than expected, the matrix needs updating.

9.2 Kappa Parameter (Likelihood Concentration)

Current state: kappa maps linearly from [0.70, 0.90] confidence to [3.0, 8.0].

What's needed:

- Test-retest reliability study: have users answer the same axis twice (separated by distractor tasks) and measure the actual noise distribution
- Calibrate kappa so that the likelihood function matches the empirical response distribution conditional on "true position"
- The linear mapping may need to be replaced with a nonlinear one if extreme positions have disproportionately higher reliability

9.3 Stopping Thresholds

Current state: Entropy floor at 20%, marginal gain threshold at 5%, minimum 5 questions.

What's needed:

- Simulate match score accuracy (comparing early-stopped profiles against full 16-axis profiles) across a range of threshold values
- Target: stopping should produce match scores within 3 percentage points of the full-information match score at least 90% of the time
- The 20% entropy floor and 5% marginal gain threshold are educated guesses that need empirical validation against this accuracy target

9.4 Imputation Confidence Mapping

Current state: Entropy reduction of 0.40 maps to confidence 0.30; 1.00 maps to 0.75 (capped).

What's needed:

- For each level of entropy reduction, compute the empirical accuracy of the imputed value (mean absolute error of imputed score vs. actual score when user later answers the axis)
- Calibrate the confidence mapping so that imputed confidence accurately reflects predictive performance
- The 0.75 cap may need to be lower (if imputation accuracy plateaus) or could be raised (if the correlation structure is stronger than expected)

9.5 Cross-Axis Propagation Damping

Current state: Damping factor equals $|\rho|$.

What's needed:

- Validate that the propagation update does not produce overconfident posteriors. Specifically: for axes with high mutual correlation (e.g., CA-EN at 0.70), check that after answering CA, the imputed EN score is correct often enough to justify the confidence level
- If propagation is too aggressive (imputed values often wrong), increase damping or cap the number of propagation hops
- Consider whether propagation should be iterative (answering A updates B, which updates C) or single-hop only. Current implementation is single-hop from the most recently answered axis. Iterative propagation risks compounding errors.

9.6 Ballot Weight Sensitivity

Current state: Ballot weights are assumed known from the ballot data pipeline.

What's needed:

- Define the ballot weight calculation: should it be binary (1.0 if axis appears on ballot, 0.0 if not) or graduated (weighted by race importance / number of candidates)?
- Test whether the sequencing order changes meaningfully with different ballot weight schemes
- Edge case: user has no ballot loaded yet (e.g., hasn't entered address). In this case, all weights should be 1.0 (uniform), and the algorithm degenerates to pure information-gain maximization.

9.7 Non-Stationarity and Response Order Effects

What's needed:

- Randomized experiment: compare adaptive ordering against fixed ordering against fully random ordering on (a) completion rate, (b) user satisfaction, and (c) profile accuracy
- Check for response order effects: does answering justice questions before economic questions produce different profiles than the reverse? If so, the algorithm may need to account for order-induced priming effects.

10. Appendix: Full TypeScript Interface Definitions

Core Types

```
// =====  
// Discrete positions and their confidence values  
// =====  
  
type DiscretePosition = -0.85 | -0.45 | 0.0 | 0.45 | 0.85;  
  
const POSITIONS: DiscretePosition[] = [-0.85, -0.45, 0.0, 0.45, 0.85];  
  
const POSITION_CONFIDENCE: Record<number, number> = {  
  0: 0.90, // position index 0 (-0.85) → high confidence
```

```

1: 0.80, // position index 1 (-0.45) → moderate confidence
2: 0.70, // position index 2 (0.00) → lower confidence
3: 0.80, // position index 3 (+0.45) → moderate confidence
4: 0.90, // position index 4 (+0.85) → high confidence
};

// =====
// Posterior distribution over one axis
// =====

interface AxisPosterior {
    probs: [number, number, number, number, number];
    mean: number;
    variance: number;
    entropy: number; // Shannon entropy in bits
}

// =====
// User value record (output per axis)
// =====

interface UserValueRecord {
    axisId: string;
    score: number; // -1 to 1
    confidence: number; // 0 to 1
    coverage_status: 'answered' | 'imputed' | 'uncovered';
    is_imputed: boolean;
    imputation_source?: string[]; // axis IDs that drove imputation
}

// =====
// Sequencing session state
// =====

interface AdaptiveSequencingState {
    /** All 16 axis IDs in canonical order */
    axisIds: string[];

    /** Current posterior distributions */
    posteriors: Map<string, AxisPosterior>;

    /** Set of directly answered axis IDs */
    answeredAxes: Set<string>;

    /** Ordered list of axis IDs as asked (for tracking) */
    questionOrder: string[];

    /** User's raw responses */
    responses: Map<string, {
        selectedScore: DiscretePosition;
        confidenceAssigned: number;
        timestamp: number;
    }>;
};

```

```

/** 16x16 correlation matrix (row-major) */
correlationMatrix: number[][];

/** Ballot-derived axis importance weights */
ballotWeights: Record<string, number>;

/** Recent domain IDs (for diversity bonus) */
recentDomains: string[];
}

// =====
// Stopping decision output
// =====

interface StoppingDecision {
  shouldStop: boolean;
  reason: 'entropy_floor' | 'marginal_gain_low' | 'all_answered' | 'continue';
  questionsAnswered: number;
  totalWeightedEntropy: number;
  maxRemainingEntropy: number;
  bestNextAxisGain: number;
}

// =====
// Imputation output
// =====

interface ImputedValue {
  axisId: string;
  score: number;
  confidence: number;
  is_imputed: true;
  coverage_status: 'imputed';
  imputation_source: string[];
  imputation_confidence_ratio: number;
}

// =====
// Complete session output (after stopping)
// =====

interface SequencingResult {
  /** All 16 axis values (answered + imputed + uncovered) */
  values: UserValueRecord[];

  /** How many questions were asked */
  questionsAsked: number;

  /** Why we stopped */
  stoppingReason: StoppingDecision['reason'];

  /** Axes that could not be imputed (entropy too high) */
  uncoveredAxes: string[];
}

```

```

/** Summary statistics */
stats: {
  meanConfidence: number;           // average confidence across all covered axes
  coveragePercent: number;          // fraction of axes that are answered or
imputed
  totalEntropyReduction: number;    // fraction of total entropy eliminated
  averageQuestionsPerDomain: number;
};
}

```

Orchestrator Function

```

/**
 * Main entry point: runs the adaptive sequencing loop.
 * In the real UI, this is split into async steps (one per question).
 * This synchronous version is for testing and simulation.
 */
function runAdaptiveSequencing(
  correlationMatrix: number[][],
  axisIds: string[],
  ballotWeights: Record<string, number>,
  /** Simulated user responses for each axis (for testing) */
  trueResponses: Map<string, DiscretePosition>
): SequencingResult {
  // Initialize state
  const state: AdaptiveSequencingState = {
    axisIds,
    posteriors: new Map(axisIds.map(id => [id, createUniformPrior()])),
    answeredAxes: new Set(),
    questionOrder: [],
    responses: new Map(),
    correlationMatrix,
    ballotWeights,
    recentDomains: [],
  };

  // Main loop
  while (true) {
    // Check stopping criterion
    const stopDecision = evaluateStopping(
      state.answeredAxes,
      state.posteriors,
      state.correlationMatrix,
      state.axisIds,
      state.ballotWeights
    );

    if (stopDecision.shouldStop) {
      return buildResult(state, stopDecision);
    }
  }
}

```

```

// Select next axis
const nextAxis = selectNextAxisWithDiversity(
  state.answeredAxes,
  state.posterior,
  state.correlationMatrix,
  state.axisIds,
  state.ballotWeights,
  state.recentDomains
);

// "Present question and get response"
const userResponse = trueResponses.get(nextAxis!);
const positionIndex = POSITIONS.indexOf(userResponse);
const confidenceAssigned = getConfidenceForPosition(positionIndex);

// Record response
state.questionOrder.push(nextAxis);
state.answeredAxes.add(nextAxis);
state.responses.set(nextAxis, {
  selectedScore: userResponse,
  confidenceAssigned,
  timestamp: Date.now(),
});

// Update posteriors
state.posterior = processUserResponse(
  state.posterior,
  nextAxis,
  userResponse,
  confidenceAssigned,
  state.correlationMatrix,
  state.axisIds
);

// Track domain for diversity
const domain = getDomainForAxis(nextAxis);
state.recentDomains.push(domain);
if (state.recentDomains.length > 2) {
  state.recentDomains.shift();
}
}
}

function buildResult(
  state: AdaptiveSequencingState,
  stopDecision: StoppingDecision
): SequencingResult {
  // Build imputed values for unanswered axes
  const imputedValues = imputeUnansweredAxes(
    state.answeredAxes,
    state.posterior,
    state.correlationMatrix,
    state.axisIds
  );

```

```

const imputedMap = new Map(imputedValues.map(v => [v.axisId, v]));

const values: UserValueRecord[] = [];
const uncoveredAxes: string[] = [];
let totalConfidence = 0;
let coveredCount = 0;

for (const axisId of state.axisIds) {
  if (state.answeredAxes.has(axisId)) {
    const response = state.responses.get(axisId)!;
    values.push({
      axisId,
      score: response.selectedScore,
      confidence: response.confidenceAssigned,
      coverage_status: 'answered',
      is_imputed: false,
    });
    totalConfidence += response.confidenceAssigned;
    coveredCount++;
  } else if (imputedMap.has(axisId)) {
    const imputed = imputedMap.get(axisId)!;
    values.push({
      axisId,
      score: imputed.score,
      confidence: imputed.confidence,
      coverage_status: 'imputed',
      is_imputed: true,
      imputation_source: imputed.imputation_source,
    });
    totalConfidence += imputed.confidence;
    coveredCount++;
  } else {
    values.push({
      axisId,
      score: 0,
      confidence: 0,
      coverage_status: 'uncovered',
      is_imputed: false,
    });
    uncoveredAxes.push(axisId);
  }
}

// Compute domain distribution
const domainCounts: Record<string, number> = {};
for (const axisId of state.questionOrder) {
  const d = getDomainForAxis(axisId);
  domainCounts[d] = (domainCounts[d] || 0) + 1;
}
const domainCount = Object.keys(domainCounts).length || 1;

const maxEntropy = computeEntropy([0.2, 0.2, 0.2, 0.2, 0.2]) *
state.axisIds.length;
let remainingEntropy = 0;

```

```

for (const axisId of state.axisIds) {
  remainingEntropy += state.posterior.get(axisId)!.entropy;
}

return {
  values,
  questionsAsked: state.questionOrder.length,
  stoppingReason: stopDecision.reason,
  uncoveredAxes,
  stats: {
    meanConfidence: coveredCount > 0 ? totalConfidence / coveredCount : 0,
    coveragePercent: coveredCount / state.axisIds.length,
    totalEntropyReduction: 1 - (remainingEntropy / maxEntropy),
    averageQuestionsPerDomain: state.questionOrder.length / domainCount,
  },
};
}

```

Summary of Design Decisions

Decision	Choice	Rationale
Prior distribution	Uniform over 5 positions	Neutrality; avoids ideological nudging
Posterior model	Discrete categorical (5-point)	Matches the structured response format exactly
Observation model	Exponential decay likelihood with kappa from confidence	Captures ordinal structure of responses
Cross-axis update	Gaussian conditional approximation with damping	Principled information transfer; damping prevents overconfidence
Selection policy	Expected Weighted Information Gain (EWIG)	Accounts for both correlation structure and ballot relevance
Diversity mechanism	10% bonus for domain switching after 2 same-domain questions	Prevents tedium without overriding information gain
Stopping rule	Entropy floor (20%) OR marginal gain < 5%, minimum 5 questions	Balances brevity with accuracy
Imputation threshold	Entropy reduction >= 40%	Conservative; only imputes when correlations provide real evidence
Imputation confidence cap	0.75 maximum	Ensures imputed axes always downweighted relative to strong direct answers
Correlation matrix	Theory-driven from ANES/Pew, awaiting empirical calibration	Best available prior; designed for update